

how do I fetch content from multiple repositories using CMake

In CMake, the standard way to fetch content from **multiple external repositories** is to use the `FetchContent` module. You simply declare each repository separately and then make them available.

Basic Pattern

cmake

✎ Copy code

```
include(FetchContent)

FetchContent_Declare(
  RepoA
  GIT_REPOSITORY https://github.com/user/repoA.git
  GIT_TAG main    # or a specific tag/commit
)

FetchContent_Declare(
  RepoB
  GIT_REPOSITORY https://github.com/user/repoB.git
  GIT_TAG v1.2.3
)

FetchContent_MakeAvailable(RepoA RepoB)
```

`FetchContent_MakeAvailable()` can take **multiple dependencies at once**.

Example with Multiple Libraries

cmake

✎ Copy code

```
cmake_minimum_required(VERSION 3.16)
project(MyProject)
```

```
include(FetchContent)

FetchContent_Declare(
    eigen
    GIT_REPOSITORY https://gitlab.com/libeigen/eigen.git
    GIT_TAG 3.4.0
)

FetchContent_Declare(
    fmt
    GIT_REPOSITORY https://github.com/fmtlib/fmt.git
    GIT_TAG 10.2.1
)

FetchContent_Declare(
    spdlog
    GIT_REPOSITORY https://github.com/gabime/spdlog.git
    GIT_TAG v1.13.0
)

FetchContent_MakeAvailable(eigen fmt spdlog)

add_executable(my_app main.cpp)

target_link_libraries(my_app
    PRIVATE
        fmt::fmt
        spdlog::spdlog
        Eigen3::Eigen
)
```

Alternative: Fetch Individually (more control)

If you need to configure each one before building:

cmake

 Copy code

```
FetchContent_Declare(RepoA ...)  
FetchContent_GetProperties(RepoA)  
if(NOT RepoA_POPULATED)  
    FetchContent_Populate(RepoA)  
    add_subdirectory(${RepoA_SOURCE_DIR} ${RepoA_BINARY_DIR})  
endif()
```

Repeat for each repo.

Best Practices

- **Pin versions:** Use a tag or commit instead of `main`
- Call `include(FetchContent)` **only once**
- Prefer `FetchContent_MakeAvailable()` unless you need customization
- Works best when the dependency supports `add_subdirectory()`

Fetching Many Dependencies Cleanly

For larger projects:

```
cmake
```

 Copy code

```
include(FetchContent)  
  
set(FETCHCONTENT_QUIET OFF)  
  
FetchContent_Declare(lib1 ... )  
FetchContent_Declare(lib2 ... )  
FetchContent_Declare(lib3 ... )  
  
FetchContent_MakeAvailable(lib1 lib2 lib3)
```

When *not* to use FetchContent

Consider alternatives if:

- Dependencies are large or slow to clone → use system packages (`find_package`)
 - You want reproducible builds across teams → use a package manager (vcpkg, Conan)
-

If you want, I can show:

- A **pattern for mixing** `find_package` + **FetchContent** (very useful), or
- A **full example for an SFML/Eigen-style project** since you've been working on C++ simulation.



what does `GIT_SHALLOW ON` mean?

`GIT_SHALLOW ON` tells CMake (via `FetchContent` or `ExternalProject`) to perform a **shallow clone** of the Git repository.

What a shallow clone is

A shallow clone downloads **only the latest snapshot** of the repository instead of the full commit history.

Equivalent Git command:

```
bash
```

 Copy code

```
git clone --depth 1 <repo-url>
```

In CMake

```
cmake
```

 Copy code

```
FetchContent_Declare(  
    mylib  
    GIT_REPOSITORY https://github.com/user/mylib.git  
    GIT_TAG v1.2.3
```

```
GIT_SHALLOW ON
)
```

What this does

- Downloads **only the specified tag/commit/branch**
 - **Does not download full history**
 - Makes downloads **much faster**
 - Uses **less disk space**
-

When to use it

Good for:

- Large repositories
 - CI builds
 - Dependencies where you **don't need history**
 - Faster configure/build times
-

Important caveats

1) Best with tags or commits

Recommended:

```
cmake
```

 Copy code

```
GIT_TAG v1.2.3
```

```
GIT_SHALLOW ON
```

Not ideal:

```
cmake
```

 Copy code

```
GIT_TAG main
```

Because shallow clones of moving branches can sometimes cause update issues.

2) Some Git operations won't work

Since history isn't downloaded:

- `git log` (full history) won't work
- Version scripts that rely on commit history may fail

3) If you need submodules

You may also need:

cmake

Copy code

```
GIT_SUBMODULES ""
```

or explicitly list them.

Summary

Option	Effect
<code>GIT_SHALLOW ON</code>	Clone with <code>--depth 1</code>
Benefit	Faster download, less space
Best practice	Use with a fixed tag or commit
Downside	No full Git history

If you're using `FetchContent` for multiple libraries, I can also show a **recommended "production" dependency block** (with shallow clone, version pinning, and offline caching).



Ok I want to use this method to build the Eigen repository with CMake

Eigen is a bit special: it's **header-only**, so you don't actually "build" it in the traditional sense. With `FetchContent`, you mainly **download it and expose its CMake target** (`Eigen3::Eigen`).

Here's the recommended way to use **FetchContent + shallow clone** for Eigen.

Minimal CMake Example (Fetch Eigen)

cmake

 Copy code

```
cmake_minimum_required(VERSION 3.16)
project(MyProject)

include(FetchContent)

FetchContent_Declare(
    eigen
    GIT_REPOSITORY https://gitlab.com/libeigen/eigen.git
    GIT_TAG 3.4.0          # Always pin a version
    GIT_SHALLOW ON
)

FetchContent_MakeAvailable(eigen)

add_executable(my_app main.cpp)

target_link_libraries(my_app
    PRIVATE Eigen3::Eigen
)
```

That's it. Since Eigen is header-only:

- No compilation step
- It just adds include directories via the `Eigen3::Eigen` interface target

Example `main.cpp`

cpp

 Copy code

```
#include <Eigen/Dense>
#include <iostream>

int main() {
    Eigen::Matrix2d A;
    A << 1, 2,
        3, 4;
    std::cout << A << std::endl;
    return 0;
}
```

Build

bash

 Copy code

```
mkdir build
cd build
cmake ..
cmake --build .
./my_app
```

What GIT_SHALLOW ON does here

Eigen's repo is large (lots of history), so shallow clone:

- Much faster download
- Smaller build directory
- No downside (Eigen doesn't need git history)

Production-quality pattern (recommended)

If you want to avoid downloading Eigen when it's already installed:

cmake

 Copy code


```
find_package(Eigen3 3.4 QUIET)

if(NOT Eigen3_FOUND)
    include(FetchContent)
    FetchContent_Declare(
        eigen
        GIT_REPOSITORY https://gitlab.com/libeigen/eigen.git
        GIT_TAG 3.4.0
        GIT_SHALLOW ON
    )
    FetchContent_MakeAvailable(eigen)
endif()
```

This:

- Uses system Eigen if available (`apt install libeigen3-dev`)
- Otherwise fetches it automatically

Common mistake

Don't do this:

```
cmake
```

 Copy code

```
include_directories(...)
```

Always use:

```
cmake
```

 Copy code

```
target_link_libraries(my_app PRIVATE Eigen3::Eigen)
```

If you're building an **N-body simulator**, I can also show a clean **CMake layout for FetchContent + Eigen + SFML** — that combination works very nicely.

    ...

How do I get a PDF of all my AI prompts and answers here?

Here are the **practical ways to export your ChatGPT conversation to PDF**, from simplest to more complete.

Option 1 — Print to PDF (Best overall)

Chrome / Edge / Firefox

1. Open the conversation you want.
2. Scroll up and down so the full conversation loads.
3. Press:
 - **Windows/Linux:** Ctrl + P
 - **Mac:** Cmd + P
4. Set:
 - **Destination** → *Save as PDF*
 - **Layout** → Portrait (usually best)
 - **More settings** → Turn **Background graphics ON** (optional)
5. Click **Save**

This captures both your prompts and the AI responses.

Tip: If the conversation is long, scroll slowly to the top first so everything loads before printing.

Option 2 — Copy into a document (clean formatting)

If you want something cleaner:

1. Select the entire conversation (Ctrl + A inside the chat pane)
2. Copy (Ctrl + C)
3. Paste into:
 - Word

- Google Docs
- Notion

4. Export:

- **File** → **Save as PDF** (Word)
- **File** → **Download** → **PDF** (Google Docs)

This gives you more control over formatting.

Option 3 — Export all ChatGPT data (full archive)

If you want **everything** you've ever asked:

1. Go to **Settings**
 2. **Data Controls**
 3. Click **Export Data**
 4. You'll receive a download link by email (ZIP)
 5. Conversations are included as **HTML/JSON**, which you can open and print to PDF.
-

Option 4 — Browser extensions (optional)

Search Chrome store for:

- "ChatGPT Export to PDF"
- "ChatGPT Conversation Exporter"

Useful if you do this often.

If you want a cleaner PDF for technical work

I can format a conversation into a **clean, structured document** (with headings, code blocks, etc.) that prints nicely.

If you'd like that, tell me:

- "Format this conversation for PDF"
- And optionally: **minimal / academic / report style**.



